`tune_diagram.py` — Documentation

A Python module for plotting accelerator tune diagrams with resonance lines and momentum-sweep trajectories. Works with both fractional tunes (e.g. `0.13`) and full tunes (e.g. `20.13`).

Quick-start example

```python
import matplotlib.pyplot as plt
from tune_diagram import TuneDiagram, SweepTrajectory

td = TuneDiagram(qx0=20.13, qy0=20.18, half_range=0.35, max_order=3, skew=True)
fig, ax = td.plot()

sweep = SweepTrajectory.from_chroma(
    qx0=20.13, qy0=20.18,
    chromax=20.18, chromay=20.13,
    delta_range=(-4e-3, 4e-3),
)
td.finalize(ax, extra_handles=[sweep.plot(ax)])
plt.show()
```

Class: `TuneDiagram`

Draws resonance lines of the form $mQ_x + nQ_y = p$ on a tune diagram, centred on a working point.

Constructor

```python
TuneDiagram(
    qx0, qy0,
    half_range=0.35,
    max_order=3,
    skew=True,
    colors=None,
    alpha=None,
    normal_ls="-",
    skew_ls="--",
    lw=1.2,
)
```

Parameter	Type	Default	Description
qx0, qy0	float	—	Working-point tunes (fractional or full).
half_range	float or (float, float)	0.35	Half-width of the axis window around the working point. A single value gives equal x/y ranges; a tuple (dx, dy) gives asymmetric ranges.
max_order	int	3	Highest resonance order to draw (inclusive).
skew	bool	True	If True, draw skew resonances ($m \neq 0$ and $n \neq 0$) with skew_ls in addition to normal resonances. If False, only normal resonances ($m = 0$ or $n = 0$) are drawn.
colors	dict	None	Override the default {order: color} mapping. Defaults: {1: "indigo", 2: "blue", 3: "yellowgreen", 4: "darkorange", 5: "red"}.
alpha	dict	None	Override the default {order: alpha} mapping. Defaults: {1: 1.0, 2: 0.85, 3: 0.70, 4: 0.55, 5: 0.45}.
normal_ls	str	"-"	Matplotlib linestyle for normal resonances.
skew_ls	str	"--"	Matplotlib linestyle for skew resonances.
lw	float	1.2	Line width for all resonance lines.

TuneDiagram.plot()

```python
fig, ax = td.plot(ax=None, figsize=(8, 7), show_working_point=True)
```

Draws all resonance lines on the axes. Call this first, then overlay sweeps, then call finalize().

Parameter	Type	Default	Description
ax	Axes	None	Existing matplotlib axes to draw on. If None, a new figure is created.
figsize	(float, float)	(8, 7)	Figure size — only used when ax=None.
show_working_point	bool	True	Plot a black marker at (qx0, qy0) and include it in the legend. Set to False when placing multiple working points manually.

Returns (fig, ax).

TuneDiagram.finalize()

```python
td.finalize(ax, legend_loc="upper left", extra_handles=None, xlabel=None, ylabel=None)
```

Adds the legend and axis labels to the plot. Call this last, after all overlays have been added.

Parameter	Type	Default	Description
ax	Axes	—	The axes to decorate.
legend_loc	str	"upper left"	Legend position (any matplotlib loc string).
extra_handles	list	None	Additional legend handles inserted at the top of the legend — typically the handle returned by sweep.plot(ax).
xlabel, ylabel	str	None	Override the default axis labels (Q_x and Q_y).

TuneDiagram.legend_handles()

```python
handles = td.legend_handles()
```

Returns the list of matplotlib.lines.Line2D handles for the resonance diagram (order colours, working-point marker, normal/skew linestyle key). Useful if you want to build a fully custom legend.

Class: SweepTrajectory

Represents a momentum-offset (δ) sweep as a curve on the tune diagram — $Q(\delta) = f(\delta)$. Do not instantiate directly; use one of the two class methods below.

SweepTrajectory.from_chroma() — linear approximation

```python
```

```
sweep = SweepTrajectory.from_chroma(
    qx0, qy0,
    chromax, chromay,
    delta_range=(-5e-3, 5e-3),
    n_points=500,
)
```

Computes the sweep using the first-order chromatic approximation:

$$Q(\delta) = Q_0 + Q' \cdot \delta$$

This assumes a straight line in tune space; the slope is set by the ratio of chromaticities.

Parameter	Type	Default	Description
`qx0`, `qy0`	`float`	—	Working-point tunes (fractional or full).
`chromax`, `chromay`	`float`	—	First-order chromaticities Q'_x, Q'_y.
`delta_range`	`(float, float)`	`(-5e-3, 5e-3)`	Momentum offset range (\delta_\min, \delta_\max) .
`n_points`	`int`	`500`	Number of sample points along the sweep.

Returns a `SweepTrajectory` instance.

`SweepTrajectory.from_twiss_scan()` — full optics scan

```python
sweep = SweepTrajectory.from_twiss_scan(
    line,
    delta_range=(-7e-3, 7e-3),
    step=None,
    n_points=None,
    delta_plot=None,
    use_fractional=False,
    verbose=True,
)
```

Computes the sweep by calling `line.twiss4d(delta0=δ)` at each δ value. The scan walks away from $\delta = 0$ in both directions, using each successfully converged closed orbit as `co_guess` for the next step — this significantly improves convergence near resonances.

Points where `twiss4d` fails are silently skipped. A `warnings.warn` message is printed for each failure if `verbose=True`, so you can see exactly which δ values were problematic.

Parameter	Type	Default	Description
`line`	xsuite `Line`	—	The lattice to twiss.
`delta_range`	`(float, float)`	`(-7e-3, 7e-3)`	Full momentum offset range to scan.
`step`	`float`	`None`	Spacing between δ points. Mutually exclusive with `n_points`.
`n_points`	`int`	`None`	Total number of δ points. Mutually exclusive with `step`.
`delta_plot`	`(float, float)`	`None`	If given, only keep points within this sub-range for plotting (e.g. the well-converged region).
`use_fractional`	`bool`	`False`	If `True`, store `qx % 1` and `qy % 1`. If `False` (default), store full tune values.
`verbose`	`bool`	`True`	Print a warning for each failed `twiss4d` call.

Returns a `SweepTrajectory` instance.

Raises `RuntimeError` if every single twiss evaluation fails.

> **Tip:** use `delta_range` wider than you need for convergence, and `delta_plot` to restrict what is actually drawn — e.g. scan `±7e-3` but only plot `±4e-3`.

`SweepTrajectory.plot()`

```python
handle = sweep.plot(ax, color="orangered", lw=2.0, ls="-", label="Sweep trajectory", zorder=3)
```

Draws the trajectory curve on `ax`.

Parameter	Type	Default	Description
`ax`	`Axes`	—	Axes to draw on.
`color`	`str`	`"orangered"`	Line colour.
`lw`	`float`	`2.0`	Line width.
`ls`	`str`	`"-"`	Linestyle.
`label`	`str`	`"Sweep trajectory"`	Legend label.
`zorder`	`int`	`3`	Drawing order (higher = on top).

Returns a `matplotlib.lines.Line2D` legend handle — pass this to `td.finalize(extra_handles=[handle])`.

Internal helpers (reference)

These are not part of the public API but are listed here for completeness.

Function	Description
`_mn_of_order(order)`	Returns all `(m, n)` pairs with `|m| + |n| == order`, with canonical sign. Non-primitive pairs (e.g. `(2, 0)`) are kept because they represent distinct resonance conditions.
`_line_key(m, n, p)`	Returns a canonical `(m, n, p)` tuple modulo positive integer scaling, used to de-duplicate lines during drawing.
`_p_values_in_box(xlim, ylim, m, n)`	Returns the range of integers `p` for which the line `m·Qx + n·Qy = p` intersects the plot box.
`_draw_line(ax, xlim, ylim, m, n, p, **kw)`	Draws a single resonance line, handling the vertical (`n=0`), horizontal (`m=0`), and general cases.
`_delta_grid(delta_range, step, n_points)`	Constructs the array of δ values for a scan, from either a step size or a point count.

Multiple working points

When comparing two working points on a single diagram, centre the `TuneDiagram` between them and use `show_working_point=False` to suppress the automatic marker and its legend entry. Then scatter the two points manually and pass their legend handles via `extra_handles`.

```python
import matplotlib.lines as mlines

fig, ax = plt.subplots(1, 1, figsize=(9, 8))

td = TuneDiagram(qx0=20.00, qy0=20.00, half_range=0.4, max_order=3, skew=True)
td.plot(ax=ax, show_working_point=False)

ax.scatter(20.13, 20.18, color="k",         zorder=5, s=40)
ax.scatter(19.87, 19.82, color="royalblue", zorder=5, s=40)

s1 = SweepTrajectory.from_twiss_scan(line=line1, delta_range=(-4e-3, 4e-3), step=1e-4)
s2 = SweepTrajectory.from_twiss_scan(line=line2, delta_range=(-4e-3, 4e-3), step=1e-4)

h_wp1 = mlines.Line2D([], [], color="k",         marker="o", ls="None", markersize=6, label="WP1 (20.13, 20.18)")
h_wp2 = mlines.Line2D([], [], color="royalblue", marker="o", ls="None", markersize=6, label="WP2 (19.87, 19.82)")
h_s1  = s1.plot(ax, color="orangered",  label="Sweep WP1")
h_s2  = s2.plot(ax, color="dodgerblue", label="Sweep WP2")

td.finalize(ax, extra_handles=[h_wp1, h_wp2, h_s1, h_s2], xlabel=r"$Q_x$", ylabel=r"$Q_y$")
plt.show()
```

Resonance condition: a resonance of order k satisfies $mQ_x + nQ_y = p$ where $|m| + |n| = k$ and p is an integer. Normal resonances have $m = 0$ or $n = 0$; skew resonances have both $m \neq 0$ and $n \neq 0$.

Fractional vs full tunes: the module works identically either way. With full tunes the integer p values are larger, but `_p_values_in_box` scans the correct range automatically.

De-duplication: because non-primitive `(m, n)` pairs are included (needed for correct order labelling), geometrically identical lines can appear from different `(m, n, p)` combinations — e.g. the line `Qx = 1` is both `(1, 0, 1)` (order 1) and `(2, 0, 2)` (order 2). The `_line_key` function ensures each line is only drawn once, at the colour of its lowest order.